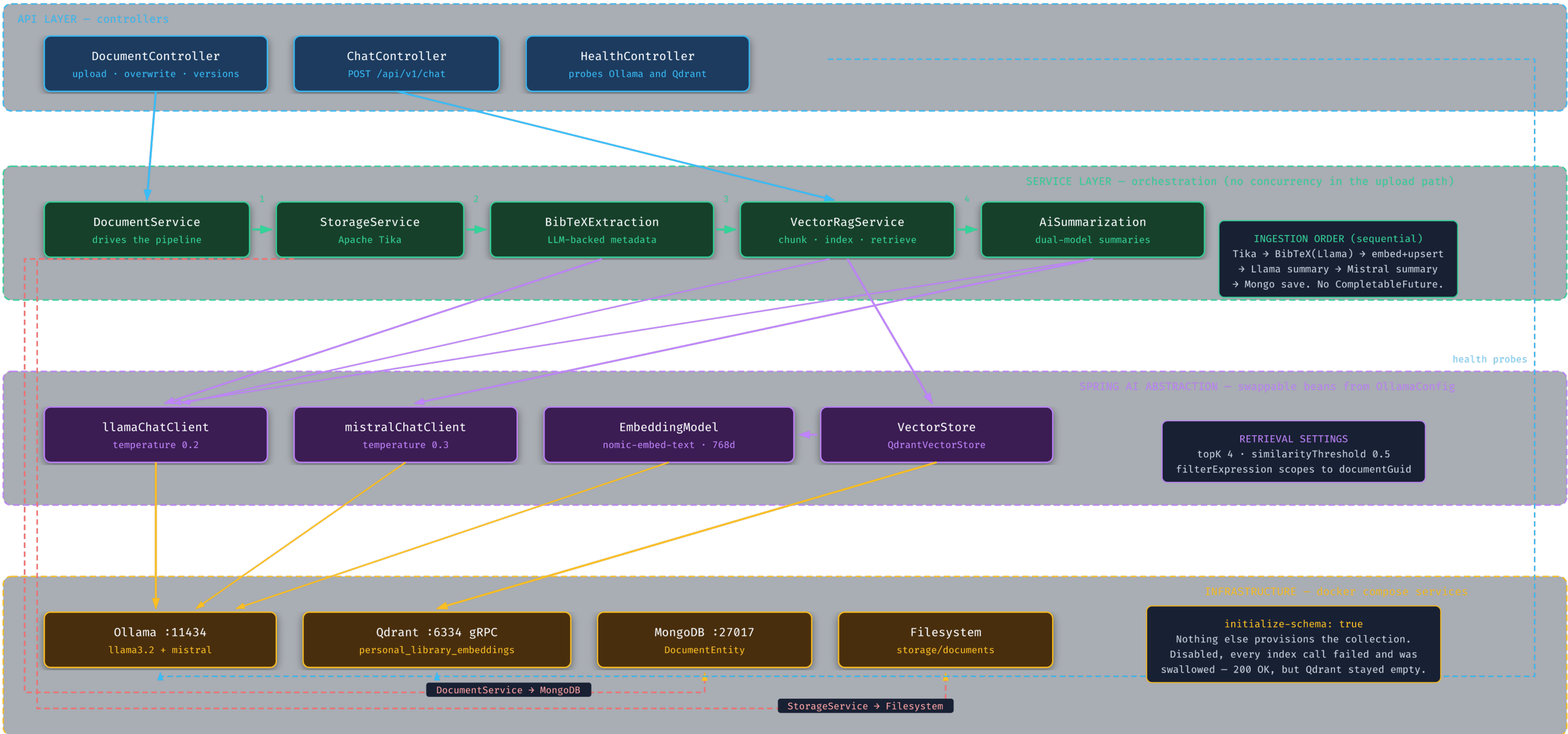


# Personal Library – Backend RAG Component Interaction

Spring AI sits between the services and the engines, so models and vector stores swap by configuration alone

Component / Data Flow



## WHY SPRING AI IS IN THE MIDDLE

The services never speak HTTP to Ollama or gRPC to Qdrant. They depend on ChatClient, EmbeddingModel and VectorStore, all built as beans in OllamaConfig. Swapping llama3.2 for a different model, or Qdrant for another store, is a config change rather than a code change.

config/OllamaConfig.java · resources/application.yml

## WHY UPLOADS ARE SLOW

One upload makes up to four Ollama round trips in series: BibTeX extraction, chunk embedding, then two summaries. Each is bounded by a 10-minute read timeout, and local CPU inference has been observed at ~2.5 minutes for a single call. The budget that matters is the SUM.

OllamaConfig.OLLAMA\_READ\_TIMEOUT · nginx 2160s · 35 min fetch

## FAILING SOFT, AND ITS COST

Every external call is wrapped in try/catch, so a dead Ollama or Qdrant never produces a 5xx. That keeps the UI usable, but it also means a broken index looks exactly like a healthy one from the browser – which is precisely how the empty-collection bug stayed hidden.

VectorRagService · HealthController exposes the real state